

Aptitude System Technical Manual

Legal Intelligence Graph Architecture

Aptitude Systems

June 3, 2026

Contents

1	Introduction	5
2	System Overview	5
3	Core Data Model	5
3.1	Legal Event Object	5
3.2	Relational Schema	5
4	Vector Retrieval Layer	6
5	API Specification	6
5.1	Submit Incident	6
5.2	Retrieve Legal Map	7
6	System Output Model	7
7	Design Principle	7
8	Legal Ontology System (Full Specification)	7
8.1	Top-Level Ontology Classes	7
8.2	Event Ontology (Misidentification / False Arrest Module)	8
8.2.1	Event Types	8
8.2.2	Event Attributes	8
8.3	Claim Ontology	8
8.4	Doctrine Ontology	9
8.5	Remedy Ontology	9
8.6	Ontology Relationship Rules	9
9	Agent Prompt System Architecture	9

10 VOLUME I — SYSTEM ARCHITECTURE AND INFRASTRUCTURE SPECIFICATION	10
10.1 Document Purpose	10
10.2 Architectural Principles	10
11 System Mission	11
12 Platform Capabilities	11
12.1 Primary Capabilities	11
13 Logical System Architecture	12
13.1 Core Service Domains	12
14 Execution Architecture	13
14.1 Workflow Lifecycle	13
15 Event Driven Architecture	13
15.1 Event Bus	13
15.2 Kafka Topic Registry	13
16 Workflow State Machine	14
17 Agent Architecture	14
17.1 Universal Agent Contract	14
17.2 Required Agent Outputs	15
18 Storage Architecture	15
18.1 Postgres Responsibilities	15
18.2 Graph Store Responsibilities	16
18.3 Vector Store Responsibilities	16
19 Database Architecture	16
19.1 Core Tables	16
20 API Architecture	17
20.1 Authentication	17
20.2 Versioning	17
21 Service Registry	17
21.1 Intake Service	17
21.2 Classification Service	17
21.3 Graph Service	18
21.4 Retrieval Service	18
21.5 Procedure Service	18
21.6 Beacon Analytics Service	18

22 Security Architecture	18
22.1 Data Classification	18
22.2 Encryption	19
22.3 Secrets Management	19
23 Audit Architecture	19
24 Observability	19
24.1 Metrics	19
24.2 Logging	19
24.3 Tracing	19
25 Disaster Recovery	20
25.1 Recovery Objectives	20
25.2 Backups	20
26 Deployment Architecture	20
26.1 Production Stack	20
27 Version Governance	20
28 System Invariants	21
28.1 Agent 1: Intake Parser Agent	21
28.2 Agent 2: Legal Classifier Agent	22
28.3 Agent 3: Graph Seeder Agent	22
28.4 Agent 4: Vector Retrieval Agent	22
28.5 Agent 5: Procedural Engine Agent	23
29 Implementation Repository Structure	23
29.1 Root Repository Layout	23
29.2 Design Principle	25
30 Execution Pipeline (Deterministic Flow)	25
31 ORCHESTRATION ENGINE SPEC	26
31.1 Execution Layer: The Nervous System	26
31.2 DATA INGESTION + LEGAL CORPUS PIPELINE: Brain Feed/Intelligence	27
31.3 Source Layers (hierarchical)	27
31.4 Step 1. Ingestion Pipeline Stages	27
31.5 Step 2: Normalize: JSON	27
31.6 Step 3: Chunking	28
31.7 Step 4: Embedding	28
31.8 Step 5: Ontology Tagging: JSON	28
32 3. Citation Graph Construction	28

33	4. Update Strategy (critical)	28
34	QUERY + RESPONSE ENGINE	29
Listings		

1 Introduction

Aptitude is a legal intelligence system designed to transform unstructured human legal experiences into structured, queryable legal knowledge.

The system operates as a multi-layer architecture consisting of:

- A structured incident intake system
- A legal knowledge graph
- A vector-based semantic retrieval layer
- A procedural reasoning engine
- A response composition layer

2 System Overview

User Narrative → Structured Event → Legal Graph Mapping → Vector Retrieval → Procedural Output
(1)

3 Core Data Model

3.1 Legal Event Object

```
{
  "event_id": "uuid",
  "event_type": "misidentification_false_arrest",
  "narrative_raw": "...",
  "jurisdiction": {
    "state": "Oregon"
  },
  "actors": [],
  "timeline": [],
  "legal_signals": {
    "rights_implicated": [],
    "procedural_triggers": []
  }
}
```

3.2 Relational Schema

```
CREATE TABLE incidents (
  id UUID PRIMARY KEY,
  raw_text TEXT,
  created_at TIMESTAMP,
  user_id UUID,
```

```
status TEXT
);
```

```
CREATE TABLE structured_events (
  id UUID PRIMARY KEY,
  incident_id UUID,
  event_json JSONB,
  event_type TEXT,
  jurisdiction TEXT
);
```

```
CREATE TABLE graph_nodes (
  id UUID PRIMARY KEY,
  node_type TEXT,
  label TEXT,
  ref_id UUID
);
```

```
CREATE TABLE graph_edges (
  id UUID PRIMARY KEY,
  from_node UUID,
  to_node UUID,
  edge_type TEXT,
  weight FLOAT
);
```

4 Vector Retrieval Layer

```
CREATE TABLE document_chunks (
  id UUID PRIMARY KEY,
  source_type TEXT,
  source_id UUID,
  chunk_text TEXT,
  embedding VECTOR(1536),
  metadata JSONB
);
```

5 API Specification

5.1 Submit Incident

```
POST /api/incident/submit
```

```
{
  "text": "...",
  "jurisdiction": "Oregon"
}
```

5.2 Retrieve Legal Map

```
GET /api/incident/{id}/result
```

6 System Output Model

- Legal classification
- Rights implicated
- Applicable statutes and case law
- Procedural enforcement pathways
- Similar prior incidents

7 Design Principle

Aptitude is not a search engine. It is a legal reasoning system that transforms lived experience into structured legal intelligence.

8 Legal Ontology System (Full Specification)

The Aptitude system is grounded in a formal ontology that defines all legal events, claims, doctrines, remedies, and procedural steps as structured, queryable entities.

This ontology enables deterministic mapping between lived experiences and legal reasoning pathways.

8.1 Top-Level Ontology Classes

The system defines six primary entity classes:

- **Event:** Real-world occurrence reported by a user
- **Claim:** Legally cognizable theory of harm
- **Doctrine:** Interpretive legal principle
- **Statute:** Codified legal rule (ORS, USC, etc.)
- **Policy:** Agency or institutional operational rule
- **Remedy:** Available legal or administrative action

8.2 Event Ontology (Misidentification / False Arrest Module)

8.2.1 Event Types

- warrant_misidentification_arrest
- eyewitness_misidentification
- database_identity_error
- biometric_false_match
- mixed_identity_detention

8.2.2 Event Attributes

Each Event must contain:

- actor_type (law enforcement, court, agency, private actor)
- identification_failure_mode
- detention_status
- probable_cause_basis
- verification_steps_taken

8.3 Claim Ontology

- false_arrest
- unlawful_detention
- due_process_violation
- negligence_identity_verification
- civil_rights_violation_42usc1983

Each claim maps to:

- required factual predicates
- burden of proof level
- jurisdictional applicability

8.4 Doctrine Ontology

- probable_cause_standard
- reasonable_suspicion_threshold
- seizure_under_fourth_amendment
- qualified_immunity_framework
- due_process_identity_verification

8.5 Remedy Ontology

- motion_to_suppress
- motion_to_dismiss
- habeas_corpus_petition
- section_1983_civil_action
- internal_affairs_complaint
- administrative_review_petition

8.6 Ontology Relationship Rules

All ontology nodes interact through deterministic edge types:

- Event $\rightarrow$ Claim: supports
- Claim $\rightarrow$ Doctrine: governed_by
- Doctrine $\rightarrow$ Statute: codified_in
- Event $\rightarrow$ Policy: violates
- Claim $\rightarrow$ Remedy: resolved_by

9 Agent Prompt System Architecture

The Aptitude system operates through a coordinated multi-agent architecture. Each agent has a strict functional boundary and deterministic output contract.

Volume I

System Architecture

Volume II

Ontology Specification

Volume III

Research Methodology

Volume IV
 Governance & Audit Framework
 Volume V
 Beacon Judicial Intelligence Framework
 Volume VI
 API and Infrastructure Specification
 graph model ontology
 ontology retrieval
 retrieval beacon analytics
 beacon analytics audit framework
 audit framework API contracts
 API contracts orchestration model
 Those dependencies need to be designed coherently.
 Volume I: System Architecture & Infrastructure Specification
 This volume would include:
 Mission and system principles
 End-to-end execution flow
 Service decomposition
 Event-driven architecture
 Kafka topics and schemas
 Workflow orchestration
 Agent contracts
 Postgres architecture

10 VOLUME I — SYSTEM ARCHITECTURE AND INFRASTRUCTURE SPECIFICATION

10.1 Document Purpose

This volume defines the authoritative architecture of the Aptitude platform.

The objective of Aptitude is to transform unstructured legal experiences, records, and questions into structured legal intelligence through deterministic workflows, auditable reasoning systems, jurisdiction-aware legal knowledge graphs, retrieval systems, and procedural analysis engines.

This document serves as the governing specification for all engineering, infrastructure, security, compliance, data governance, and product development activities.

10.2 Architectural Principles

1. Auditability Before Automation
2. Structured Truth Before Model Output
3. Reproducibility Before Performance
4. Jurisdictional Isolation

5. Explainability By Design
6. Event-Driven Execution
7. Immutable Historical Records
8. Human Review For High-Risk Outputs
9. Versioned Knowledge
10. Deterministic System Behavior

11 System Mission

Aptitude exists to:

- Structure legal experiences
- Map experiences to legal concepts
- Retrieve authoritative legal material
- Generate procedural intelligence
- Maintain complete auditability
- Preserve evidentiary provenance
- Surface uncertainty explicitly

12 Platform Capabilities

12.1 Primary Capabilities

1. Incident Intake
2. Fact Structuring
3. Legal Classification
4. Ontology Mapping
5. Citation Graph Construction
6. Semantic Retrieval
7. Procedural Analysis
8. Judicial Analytics
9. Legal Research Support

10. Document Intelligence
11. Evidence Intelligence
12. Compliance Monitoring
13. Audit Generation

13 Logical System Architecture

User Layer

↓

API Gateway

↓

Workflow Orchestrator

↓

Agent Services

↓

Knowledge Services

↓

Storage Layer

↓

Observability Layer

13.1 Core Service Domains

1. Intake Domain
2. Classification Domain
3. Graph Domain
4. Retrieval Domain
5. Research Domain
6. Procedure Domain
7. Analytics Domain
8. Governance Domain
9. Audit Domain

14 Execution Architecture

14.1 Workflow Lifecycle

Each user interaction becomes a workflow.

```
workflow_id = uuid()

incident_id = uuid()

correlation_id = uuid()
```

All actions are linked through:

- workflow_id
- incident_id
- correlation_id
- jurisdiction_id

15 Event Driven Architecture

15.1 Event Bus

Recommended:

1. Apache Kafka (Production)
2. Redpanda (Alternative)
3. Redis Streams (MVP)

15.2 Kafka Topic Registry

```
incident.created
incident.validated
incident.structured
incident.classified
graph.nodes.created
graph.edges.created
retrieval.requested
retrieval.completed
research.completed
procedure.generated
beacon.analysis.completed
response.generated
```

```
response.delivered
audit.recorded
workflow.failed
workflow.replayed
```

16 Workflow State Machine

```
START
↓
INGESTION
↓
VALIDATION
↓
STRUCTURING
↓
CLASSIFICATION
↓
GRAPH EXPANSION
↓
RETRIEVAL
↓
RESEARCH SYNTHESIS
↓
PROCEDURAL ANALYSIS
↓
RESPONSE ASSEMBLY
↓
AUDIT RECORDING
↓
COMPLETE
```

17 Agent Architecture

17.1 Universal Agent Contract

```
class Agent:

    ...

    def run(
        self,
```

```
    event: dict,  
  
    state: dict  
  
    ) -> AgentResult:  
  
    pass  
  
    ...
```

17.2 Required Agent Outputs

Every agent returns:

- output
- confidence
- audit_metadata
- emitted_events
- version
- execution_metrics

18 Storage Architecture

18.1 Postgres Responsibilities

Canonical data storage.

Stores:

- incidents
- events
- classifications
- citations
- audits
- users
- permissions
- workflow states

18.2 Graph Store Responsibilities

Stores:

- legal concepts
- doctrines
- statutes
- authorities
- relationships
- judicial analytics

Recommended:

Neo4j

or

Memgraph

18.3 Vector Store Responsibilities

Stores:

- embeddings
- chunk metadata
- citation metadata
- retrieval metrics

Recommended:

pgvector

or

Qdrant

19 Database Architecture

19.1 Core Tables

incidents

structured_events

classifications

graph_nodes

graph_edges

document_registry

document_chunks

citation_registry
workflow_state
audit_log
agent_executions
beacon_scores
judge_profiles
research_runs

20 API Architecture

20.1 Authentication

OAuth2
JWT
API Keys
Service Accounts

20.2 Versioning

All endpoints versioned.

Example:
/api/v1/...

21 Service Registry

21.1 Intake Service

Responsibilities:

- accept user submissions
- validate schemas
- create workflows

21.2 Classification Service

Responsibilities:

- ontology matching
- claim identification
- doctrine mapping

21.3 Graph Service

Responsibilities:

- graph construction
- edge creation
- graph traversal

21.4 Retrieval Service

Responsibilities:

- semantic search
- authority retrieval
- citation retrieval

21.5 Procedure Service

Responsibilities:

- procedural path generation
- prerequisite detection
- deadline analysis

21.6 Beacon Analytics Service

Responsibilities:

- judicial analytics
- behavioral metrics
- trend analysis
- score computation

22 Security Architecture

22.1 Data Classification

PUBLIC

INTERNAL

CONFIDENTIAL

RESTRICTED

22.2 Encryption

AES-256 at rest
TLS 1.3 in transit

22.3 Secrets Management

Vault
AWS Secrets Manager
Azure Key Vault

23 Audit Architecture

Every system action must record:

- actor
- timestamp
- source data hash
- output hash
- version
- jurisdiction
- workflow id

24 Observability

24.1 Metrics

Workflow latency
Agent latency
Retrieval latency
Classification accuracy
Audit completeness

24.2 Logging

Structured JSON logs only.

24.3 Tracing

OpenTelemetry required.

25 Disaster Recovery

25.1 Recovery Objectives

RPO < 15 minutes

RTO < 4 hours

25.2 Backups

Daily

Weekly

Monthly

Immutable archive copies

26 Deployment Architecture

26.1 Production Stack

Kubernetes

Kafka

Postgres

Neo4j

Qdrant

Redis

FastAPI

OpenTelemetry

Prometheus

Grafana

27 Version Governance

Every artifact must be versioned.

Includes:

- prompts
- ontology
- retrieval models
- embeddings
- graph schema
- API contracts
- workflow definitions
- beacon algorithms

28 System Invariants

The following rules may never be violated:

1. No output without provenance.
2. No citation without source.
3. No authority without jurisdiction.
4. No overwrite of historical records.
5. No deletion of audit trails.
6. No legal conclusion without supporting authorities.
7. No procedural recommendation without prerequisite validation.
8. No score without reproducible methodology.
9. No model output without version tracking.
10. No workflow execution without audit recording.

Vector store architecture

Graph architecture

Security model

Audit model

Explainability model

API contracts

Deployment architecture

Kubernetes topology

Observability

Disaster recovery

Versioning strategy

28.1 Agent 1: Intake Parser Agent

SYSTEM ROLE:

You are a legal event structuring engine.

TASK:

Extract structured legal facts from unstructured narrative.

OUTPUT REQUIREMENTS:

Return valid JSON matching the Legal Event schema.

RULES:

- Do not infer legal conclusions

- Do not provide advice
- Only structure observable facts
- Preserve uncertainty explicitly

28.2 Agent 2: Legal Classifier Agent

SYSTEM ROLE:

You are a legal classification engine.

TASK:

Map structured events to legal claim types and doctrine clusters.

OUTPUT:

```
{
  "claims": [...],
  "doctrines": [...],
  "confidence_scores": {...}
}
```

RULES:

- Use ontology only
- Do not hallucinate statutes or cases
- Prefer conservative classification

28.3 Agent 3: Graph Seeder Agent

SYSTEM ROLE:

You are a legal knowledge graph constructor.

TASK:

Attach legal nodes (statutes, cases, policies, remedies) to structured events.

OUTPUT:

Graph edges in standardized format.

RULES:

- Every edge must have a defined relationship type
- No orphan nodes allowed
- Jurisdiction filtering is mandatory

28.4 Agent 4: Vector Retrieval Agent

SYSTEM ROLE:

You are a semantic legal retrieval system.

```
TASK:
Retrieve top-k relevant:
- cases
- statutes
- agency policies

METHOD:
Embedding similarity search

OUTPUT:
Ranked document chunks with metadata

RULE:
Return only supporting material, not conclusions
```

28.5 Agent 5: Procedural Engine Agent

```
SYSTEM ROLE:
You are a legal procedural mapping engine.

TASK:
Generate enforcement pathways based on claim types.

OUTPUT:
Structured procedural trees including:
- criminal defense options
- civil remedies
- administrative processes

RULES:
- Must align with jurisdiction
- Must include procedural ordering
- Must not omit prerequisite steps
```

29 Implementation Repository Structure

The system is implemented as a modular distributed architecture.

29.1 Root Repository Layout

```
backend/
  api/
    routes/
    controllers/
```

```
    middleware/

services/
    ingestion_service/
    classification_service/
    graph_service/
    retrieval_service/
    procedural_service/

agents/
    intake_parser_agent.py
    legal_classifier_agent.py
    graph_seeder_agent.py
    vector_retrieval_agent.py
    procedural_engine_agent.py

models/
    incident.py
    legal_event.py
    graph_models.py

db/
    postgres/
    migrations/
    vector_store/

event_bus/
    producers.py
    consumers.py
    event_schemas.py

main.py

frontend/
    app/
    components/
    pages/
    graph_viewer/
    api_clients/

ontology/
    event_types.json
    claims.json
    doctrines.json
    remedies.json

pipelines/
    ingestion_pipeline.py
    embedding_pipeline.py
    graph_build_pipeline.py
```



```
infra/  
  docker/  
  kubernetes/  
  redis/  
  postgres/  
  
docs/  
  system_architecture.tex  
  ontology_spec.tex  
  api_spec.tex  
  
tests/  
  unit/  
  integration/  
  agent_tests/
```

29.2 Design Principle

The system is explicitly modular:

- Agents are stateless and replaceable
- Graph structure is deterministic and auditable
- LLM usage is strictly constrained to classification and retrieval assistance
- Legal truth is stored in structured databases, not in model weights

30 Execution Pipeline (Deterministic Flow)

- **Step 1: Ingestion**
 - raw_text stored in incidents table
 - incident_id generated
- **Step 2: Structuring**
 - Intake Parser Agent outputs Legal Event JSON
 - persisted in structured_events
- **Step 3: Classification**
 - Legal Classifier assigns claims + doctrines
 - outputs appended to event_json
- **Step 4: Graph Expansion**
 - Graph Seeder converts event + classification into nodes/edges

- writes to graph_nodes and graph_edges
- **Step 5: Retrieval**
 - Vector Retrieval Agent queries document_chunks
 - returns ranked legal context
- **Step 6: Reasoning Assembly**
 - Procedural Engine constructs enforcement pathways

31 ORCHESTRATION ENGINE SPEC

Defines:

- how agents are actually executed
- how data flows between them
- how graph + vector + SQL stay in sync

31.1 Execution Layer: The Nervous System

Core Principle The system is event-driven + stateful + replayable.

Every user input becomes:

IncidentCreated → EventSourcedPipeline → GraphUpdated → RetrievalExecuted → ResponseBuilt

No agent runs “freely.” Everything is triggered by events.

2. Event Bus Architecture Technology assumption Kafka (preferred) OR Redis Streams (lightweight MVP)

3. Canonical Event Types YAML IncidentCreated IncidentStructured EventClassified GraphNodesCreated GraphEdgesCreated VectorRetrievalRequested VectorRetrievalCompleted ProceduralPlanGenerated ResponseFinalized PipelineFailed

4. Orchestration Model (Temporal-style state machine)

Each incident has a Workflow ID

workflow_id = incident_id

State Machine:

START ↓ INGESTION ↓ STRUCTURING ↓ CLASSIFICATION ↓ GRAPH_EXPANSION
↓ VECTOR_RETRIEVAL ↓ REASONING_ASSEMBLY ↓ RESPONSE_COMPOSITION
↓ COMPLETE

5. Execution Contract per Agent

Each agent MUST implement (python)

```
def run(input_event: dict, state: dict) -> dict: return "output": ..., "new_state": ..., "emitted_events": [...]
```

No exceptions.

6. Failure Handling

Every stage has:
retry (3x exponential backoff) dead-letter queue partial state persistence
PipelineFailed → store state snapshot → log failure reason → allow manual
replay

7. Auditability Requirement

Every transformation must store:

input hash output hash agent version timestamp jurisdiction context

This is non-negotiable if this touches legal material.

31.2 DATA INGESTION + LEGAL CORPUS PIPELINE: Brain Feed/Intelligence

31.3 Source Layers (hierarchical)

Tier 1: Binding law ORS / USC / CFR

state administrative codes

Tier 2: Case law CourtListener API

state appellate databases

Supreme Court opinions

Tier 3: Agency systems (critical for your use case) DHS manuals

police policy manuals

probation/parole guidelines

internal SOPs

Tier 4: Secondary interpretive material law reviews

training guides

complaint templates

31.4 Step 1. Ingestion Pipeline Stages

Step 1: Fetch source_url → raw_document_st

31.5 Step 2: Normalize: JSON

Convert everything into:

```
"doc_id": "", "source_type": "statute | case | policy | manual", "jurisdiction":  
"", "effective_date": "", "text": ""
```

31.6 Step 3: Chunking

Chunk rules:

- 300–800 tokens per chunk
- preserve section boundaries
- attach metadata per chunk

31.7 Step 4: Embedding

chunk → embedding_model → vector

Store in:

document_chunks (id, doc_id, chunk_text, embedding, citations, jurisdiction, source_type)

31.8 Step 5: Ontology Tagging: JSON

Each chunk gets labels:

- doctrine tags
- claim relevance
- procedural relevance

Example:

"tags": ["false_arrest", "probable_cause", "fourth_amendment"]

32 3. Citation Graph Construction

You also build a citation network graph:

- case cites case
- statute interprets statute
- policy derives from statute

This becomes:

citation_graph_edges

This is what lets your system behave like Lexis—but structured.

33 4. Update Strategy (critical)

Law changes. So:

- version every document
- never overwrite
- maintain temporal validity window

Defines:

- how DHS / statutes / cases enter system
- normalization rules
- chunking + citation linking

34 QUERY + RESPONSE ENGINE

Defines:

- how a user question becomes graph traversal + retrieval + reasoning output
- response formatting model (legal memo style, dashboard, etc.)